

Exploring Gender Bias in Word2Vec

Jun 10, 2021

I've always found the idea of word vectors fascinating. Word vectors (also called word embeddings) turn a word into an array of numbers, and makes it possible to do *math on words*. For instance, you can [add and subtract words from each other](#) to get new words, like "Tokyo" - "Japan" + "France" $\approx$ "Paris". Less ideally, they also learn the biases about words that are present in the text the word vectors were trained from, as famously expressed in the paper [Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings](#). This paper showed how it's possible to mathematically analyze the gender bias in word vectors and see which words have a male or female bias.

The paper presents ideas to remove the bias from word vectors, but I was curious instead to use what the word vectors had learned to try to understand the gender bias in words in English. I created a project called [Word2Vec Gender Bias Explorer](#) which lets you browse the gender bias in any words you enter as present in the original [Google News Word2Vec dataset](#). In this article, I'll go through how this works, and the problems I ran into building the explorer. This project uses Word2Vec word vectors, but it should also work just as well with other word vector types like Glove.

Detecting gender bias in word vectors

The idea for finding bias in word vectors comes from first finding a "gender" dimension in the data. This is done by subtracting words that are known to be male from their equivalent female version. For example, word pairs that could be used are things like girl - boy, queen - king, mother - father, etc... In each of these cases, the words are nearly identical in all ways except for the gender they refer to. As such, subtracting these words should result in a vector that mostly represents the idea of "gender". In order to find a single "gender" vector from all of these pair differences, you can just average all the individual gender vectors together. Or, for even more accuracy, [principle component analysis](#), or PCA, can be used to find the main vector that all of these pair differences share (more on this later). To start, we'll use with the averaging technique since it's simpler:

```
import numpy as np

gender_pairs = [
    ("girl", "boy"),
    ("queen", "king"),
    ...
]
gender_vectors = [
    word_vectors[pair[0]] - word_vectors[pair[1]]
    for pair in gender_pairs
]
gender_vector = np.mean(gender_vectors, axis=0)
```

Once a "gender" vector is found, each word in the "male" set and the "female" set is project along this direction (projecting just means taking the dot product of the "gender" vector with the "word" vector). For instance, to find the "gender" projection for "queen", we'd just do something like:

```
import numpy as np

queen_gender_projection = np.dot(word_vectors["queen"], gender_vector)
```

Then, we can do this for each word in the "male" group, and each word in the "female" group, and get an average gender projection for each group:

```
male_projections = [np.dot(word_vectors[word], gender_vector) for word in male_words]
female_projections = [np.dot(word_vectors[word], gender_vector) for word in female_words]

mean_male_projection = np.mean(male_projections)
mean_female_projection = np.mean(female_projections)
```

Now, to see if a word has a male or female gender bias, we can do this same process and see if it lies closer to the "male" mean or the "female" mean:

```
test_word = "nurse"
test_word_projection = np.dot(word_vectors[test_word], gender_vector)

mean_projection = (mean_male_projection + mean_female_projection) / 2
# scale the score so > 0 means female bias, < 0 means male bias
test_word_score = 2 * (test_word_projection - mean_projection) / (mean_female_projection - mean_male_projection)
```

Using PCA to find the gender vector

Above we just average all the difference from the gendered word pairs together to get an overall "gender" vector, but we can get a better vector by using PCA to find the main component that all these gendered pairs have in common. PCA works by trying to break down a group of vectors into the main components that account for the most variation between the vectors.

Getting PCA to work here is a bit tricky, since if we just run PCA on all the biased words directly we won't get something resembling a gender vector. We need to make sure that the largest variation among all the data points we're using should be the gender variation, but just comparing things like "boy" and "queen" will have lots of huge variations in other dimensions too. If we just take all the difference vectors and run PCA against them we also won't get a gender vector. The gender difference vectors will all be pretty close to each other, so PCA is just going to pick up on noise by looking at these difference vectors all in a group.

To get this to work, I created an additional set of difference vectors by taking the `male_word - female_word` for each word pair, in addition to the reverse `female_word - male_word`. This should really exacerbate the gender difference so PCA can pick up on it when it calculates the vector accounting for the most variation between the vectors passed in. I'm not 100% sure this is the correct way to do things, but I think it should be valid. (If I'm doing something wrong here, please let me know!)

Doing a PCA fit on the data is easy using [Scikit Learn](#), since it has a PCA module which does the PCA transform with just a few lines of code. The code to find the gender vector using PCA is below:

```
from sklearn.decomposition import PCA

gender_vectors = []
for (female_word, male_word) in gender_pairs:
    # try to exacerbate the gender direction by adding both positive and negative versions
    # of each gender difference vector
    gender_vectors.append(female_word - male_word)
    gender_vectors.append(male_word - female_word)

pca = PCA(n_components=1)
pca.fit(np.array(gender_vectors))

mean_female_projection = np.mean(
    pca.transform(np.array([pair[0] for pair in gender_pairs]))
)
mean_male_projection = np.mean(
    pca.transform(np.array([pair[1] for pair in gender_pairs]))
)
```

You'll notice that above there's no explicit `gender_vector` in the above code - instead, running `pca.transform(word_vector)` has the effect of projecting the `word_vector` in the gender direction all in one go. From here, the calculation of where a word vector's bias lies is the same as above:

```
test_word = "nurse"
test_word_projection = pca.transform(np.array([word_vectors[test_word]]))[0][0]

mean_projection = (mean_male_projection + mean_female_projection) / 2
# scale the score so > 0 means female bias, < 0 means male bias
test_word_score = 2 * (test_word_projection - mean_projection) / (mean_female_projection
```

Wait, why do most words have a male bias?

After doing this analysis, it turns out that the vast majority of words score something like ~70% male. These include words that make no sense to have a bias, words like "the" or "and", "where", etc... I suspect that this is because most of the Google News dataset that the word vectors are trained from probably talk about men a lot more often than they talk about women, so the word vectors probably learned that most words are more likely to occur near male words on average. I'm not completely sure if this is correct though, so if I'm overlooking something please let me know!

Regardless, I adjusted the model to account for this by taking some words that are definitionally neutral, like "the" or "and", "where", etc and calculating a mean projection for those words as well, just like we did above for the mean "male" and mean "female" projections. Then, I just rescaled everything to force the "neutral" mean to map to 0, the "male" mean to map to -1, and the "female" mean to map to 1. This seems to work pretty well to avoid everything showing up as "male". Most words are roughly neutral, which makes intuitive sense, but ironically the word "computer programmer" from the original paper also maps to roughly neutral after doing this!

Some words have strange biases

Most words have a bias that makes sense, but there are a few words that seems to have a very strange bias result after doing this analysis. The most prominent was the word "husband", which this analysis says is a female biased word. Interestingly, "wife" also shows up as female biased, but that makes a lot more sense. I suspect that "husband" is showing up as female since probably most articles where the word "husband" appeared were surrounded by a strongly female word, like "her husband" or "Janet's husband", etc... Since word2vec is trained on a model that tries to predict words near the word in question, it seems feasible that if the words surrounding "husband" usually have a female word like "her", that it might learn that "husband" is female. Again, this is just my speculation, so let me know if I'm overlooking something!

Go forth and explore!

The final result is online at chanind.github.io/word2vec-gender-bias-explorer, and you can see all the code at github.com/chanind/word2vec-gender-bias-explorer. If you have any ideas to improve the code please open an issue or submit a pull request. Contributions and improvements are always welcome!

ALSO ON CHANIND.GITHUB.IO

On Building Wechat Miniprograms 6 years ago · 2 comments Wechat miniprograms are extremely popular in China right now, so I've been ...	Your Jest Tests are Leaking Memory 6 years ago · 9 comments Jest is designed in a way that makes memory leaks likely if you're not actively ...	Manipulating and Animating SVG with ... 7 years ago · 1 comment When I first started building the Hanzi Writer library, I assumed that in order to ...	Why I miss R 6 years ago · 45 The world of t fast, and there frameworks ar
---	---	--	--

0 Comments

Login ▾



Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name



Share

Best

Newest

Oldest

Be the first to comment.

[Subscribe](#)

[Privacy](#)

[Do Not Sell My Data](#)

chanind.github.io

chanind.github.io
chanindav@gmail.com



Machine Learning, Javascript, Python, and
DevOps